From Discovery to Deployment: An Embedded RISC-V Evaluation of Kolmogorov–Arnold Networks in Hard-Constrained Recurrent Physics-Informed Models

Enzo Nicolás Spotorno, Josafat Leal Filho, Antônio Augusto Fröhlich
Department of Informatics and Statistics
Federal University of Santa Catarina, Florianópolis, Brazil
{enzoniko, josafat, guto}@lisha.ufsc.br

***Abstract*—Hard-constrained recurrent physics-informed networks (HRPINNs) embed known dynamics in a recurrent integrator and delegate only the unmodeled residual to a small neural branch, making them natural candidates for real-time digital twins on resource-constrained cyber-physical systems (CPS). Kolmogorov–Arnold Networks (KANs) have been proposed as parameter-efficient replacements for MLPs in such residual branches, but their learnable B-spline activations have a fundamentally different execution profile. We present, to our knowledge, the first systematic embedded evaluation of the vanilla B-spline KAN as an HRPINN residual branch: a zero-dependency, scalar C++17 inference engine with interchangeable KAN and MLP branches, bit-verified against the training framework and deployed on a RISC-V RV64GC platform without vector extensions (StarFive VisionFive 2, SiFive U74). Building on algorithmic results that established when KANs match MLPs in discovery accuracy, we evaluate three systems-level hypotheses: execution latency (H1), energy per integration step (H2), and dependability under post-training quantization in the closed recurrent loop (H3). On identical trained models, the KAN residual branch executes $2.9\times$–$4.5\times$ slower than an accuracy-comparable MLP at similar parameter count ($3.7\times$ under a deployment-optimized basis evaluation), and exhibits input- and weight-dependent latency variation of up to $2.2\times$ that MLPs do not show. Under INT8 quantization in the recurrent loop, KAN trajectories diverge up to $43\times$ earlier than matched MLPs, and attribution experiments show the dominant mechanism is per-tensor quantization of heterogeneous spline weights rather than the often-cited grid mis-binning. [TBD: Board latency and μJ-per-step energy figures from the VisionFive 2 measurement session.] We distill the results into deployment guidelines for physics-informed edge CPS.**

***Index Terms*—Kolmogorov–Arnold networks, physics-informed learning, embedded AI, RISC-V, quantization, dependability, cyber-physical systems, digital twins**

I. Introduction

Modern cyber-physical systems increasingly rely on executable models of their own physics — digital twins — for prediction, anomaly detection, and control. When such models must run *on* the system, under real-time and energy budgets, two properties become essential: physical consistency by construction, and a computational footprint compatible with low-cost edge silicon. Hard-constrained recurrent architectures such as the Hybrid Recurrent Physics-Informed Neural Network (HRPINN) [1] address the first property structurally: the known dynamics and the numerical integrator are fixed inside the recurrent update rule, and a neural branch learns *only* the residual dynamics that the first-principles model does not capture. This design was demonstrated to enforce physical invariants in CPS settings and, importantly for this work, it reduces the learned component to a small function $R_\theta : \mathbb{R}^n \to \mathbb{R}^m$ evaluated once per integration step — precisely the granularity at which embedded inference cost is decided.

Kolmogorov–Arnold Networks (KANs) [2] recently emerged as an alternative to MLPs for scientific machine learning. Grounded in the Kolmogorov–Arnold representation theorem, they replace fixed node activations with learnable univariate B-splines on edges, offering an additive inductive bias that aligns well with physical laws expressed as sums of low-order nonlinear terms, and have shown promise for gray-box recovery of hidden terms in dynamical systems [3], [4], [5], [6]. In prior algorithmic work, summarized in Section II, we established under which conditions the *vanilla* B-spline KAN matches or loses to MLPs as an HRPINN residual branch in discovery accuracy and training stability.[1]

Accuracy, however, is only half of the deployment equation. The literature consistently reports that KAN inference is expensive relative to its parameter count: recursive B-spline evaluation consumes many hardware resources, and even LUT-based simplifications require substantial multiplexing and decoding logic [7]; analytical operation counts attribute to the original KAN formulation a several-fold multiplication overhead relative to equally-sized MLPs [8]; and hardware implementations of the basis functions are identified as a primary efficiency burden [9]. Yet these observations come from accelerator-design studies of KANs as generic classifiers/regressors. What is missing — and what a CPS engineer choosing a residual-branch architecture actually needs — is a controlled, end-to-end measurement on the deployment class

[1]Preliminary algorithmic results were presented at a non-archival ICLR 2026 workshop (no DOI); this paper subsumes them, adds the embedded systems evaluation, and is the archival reference.

that hybrid physics-informed models target: a scalar embedded core, inside a recurrent integrator, with the accuracy of each measured model known on the same task. That bridge between the algorithmic result and the systems result, on *identical trained weights*, is the contribution of this paper.

A. Why this evaluation, why now, and why on these systems

Three scoping decisions deserve explicit justification.

Why the vanilla KAN. Dozens of KAN variants (Chebyshev, wavelet, operator-network, hybrid) now exist [10], [11], [12], each claiming efficiency or stability improvements. Every one of them retains the defining computational primitive — per-edge univariate function evaluation followed by summation — and most retain B-splines. A rigorous hardware baseline of the foundational formulation is therefore a prerequisite: it isolates the cost of the *paradigm* from the cost of any particular engineering of it, and gives later variant evaluations a common reference point. This mirrors standard practice in architecture research, where the canonical algorithm is characterized before its optimized descendants.

Why embedded in HRPINN, on a scalar RISC-V core. The residual branch is the only learned component of HRPINN, so the architecture comparison is perfectly isolated: swapping R_θ changes nothing else in the computation. HRPINN's stated purpose is dependable prediction *on* CPS platforms [1], so per-step latency and energy on edge silicon are not an afterthought but the operating point the architecture was designed for. We target the StarFive VisionFive 2 (SiFive U74, RV64GC): it lacks the RISC-V vector extension, so both MLP matrix products and KAN spline recursions compile to pure scalar instruction streams. This is deliberate: scalar execution exposes the *architectural* overhead of each topology (operation mix, branching, locality) rather than the quality of vendor kernels, and it is representative of the low-cost industrial SoCs where physics-informed digital twins are expected to run. RISC-V additionally makes the platform fully open and reproducible [13].

Why canonical oscillators rather than a complex industrial case study. We evaluate on the Duffing and Van der Pol oscillators — the same systems as the algorithmic study — for four methodological reasons. (i) *Analytical ground truth*: both residuals are known in closed form ($-0.3x^3$; $(1-x^2)v$), so discovery accuracy (R^2 against the true residual manifold) is exactly computable, which is what allows every hardware number in this paper to be paired with the accuracy of the very model being measured; no real-world plant offers this. (ii) *Representative structure*: second-order oscillatory dynamics are the canonical abstraction of the mechanical and electrical subsystems where HRPINN-class twins apply (suspensions, vibrating machinery, power converters, PLLs), and the pair spans the axis that governs KAN behavior — additively separable (Duffing) versus multiplicative state coupling (Van der Pol). (iii) *Confound isolation*: with a complex plant, identification error, unmodeled disturbances, and data quality would confound hardware-induced degradation; with a controlled system, any closed-loop divergence under quantization is attributable to the arithmetic, which is the object of study. (iv) *A fortiori scaling*: the per-step cost of both architectures is an explicit, exactly countable function of layer widths (Table II), so costs extrapolate predictably to higher-dimensional states, while failure modes only compound — an architecture that misses latency, energy, or dependability targets on the simplest two-state benchmark is disqualified for harder plants; conversely, nothing about a larger plant removes the per-edge spline machinery measured here. The simplification is thus conservative with respect to our conclusions. Complex-plant case studies are the natural next step once the architectural question is settled, and we outline them in Section VII.

B. Contributions

- A lightweight, zero-dependency, scalar-optimized C++17 deployment engine for HRPINNs with interchangeable MLP and B-spline KAN residual branches, bit-verified against the PyTorch training stack through golden-vector test suites (35 tests / 11k checks, identical on x86-64 and RV64GC), and released with all models, golden data, and measurement scripts.
- A controlled latency and energy evaluation on a RISC-V RV64GC SoC, framing and testing explicit hypotheses (H1, H2) about the scalar execution cost of the vanilla KAN, including an anti-strawman variant with deployment-optimized (precomputed-reciprocal) basis evaluation and analytical operation counts for every measured model.
- A recurrent dependability analysis (H3) of post-training static quantization (INT16/8/6/4) inside the closed HRPINN loop, with attribution experiments (input-, weight-, activation-only quantization) that identify *which* quantization point breaks each architecture, and a quantized-precision extension of the discovery-accuracy metric that bridges the algorithmic and systems results.
- Deployment guidelines for residual-branch selection in embedded physics-informed CPS.

II. Background and Algorithmic Foundations

A. HRPINN and the residual-learning setting

HRPINN advances the normalized state $s_t = [x_t, v_t]$ with a hard-constrained explicit-Euler cell,

$$\begin{aligned} \dot{v}_t &= a_{\text{known}}(x_t, v_t) + R_\theta(s_t), \\ x_{t+1} &= x_t + \Delta t\, v_t, \qquad v_{t+1} = v_t + \Delta t\, \dot{v}_t, \end{aligned} \tag{1}$$

with $\Delta t = 0.05\,\text{s}$. The known physics a_{known} and the integrator are fixed; only R_θ is trained, either by one-step teacher forcing (TF) or by backpropagation through time (BPTT) over 20-step windows. The residual branch is implemented either as a ReLU MLP or as a vanilla B-spline KAN [2] (grid size G=5, order k=3 unless stated; SiLU base path plus spline path, with training-adaptive, generally non-uniform grids). Benchmark systems: Duffing ($\dot{v} = -x - 0.2v - 0.3x^3$, known part $-x - 0.2v$, residual $-0.3x^3$) and Van der Pol ($\dot{v} = -x + (1-x^2)v$, known part $-x$, residual $(1-x^2)v$).

TABLE I
ALGORITHMIC BASELINE (BPTT RECIPE): DISCOVERY R^2, MEAN $\pm$95% CI OVER 100 SEEDS, FROM THE PRELIMINARY STUDY. PARAMS AS IN TABLE II.

Arch.	Config.	Duffing R^2	Van der Pol R^2
KAN	Very Small	0.914 ± 0.061	0.743 ± 0.061
	Small	0.874 ± 0.080	0.785 ± 0.073
	Wide	0.468 ± 0.773	-0.602 ± 2.842
	Deep	(unstable)	0.754 ± 0.079
MLP	Tiny	0.906 ± 0.092	0.622 ± 0.173
	Small	0.937 ± 0.047	0.879 ± 0.032
	Medium	0.951 ± 0.033	0.879 ± 0.019
	Large	0.932 ± 0.063	0.898 ± 0.017

Discovery accuracy is the R^2 between the learned surface $R_\theta(x, v)$ and the analytical residual on a 50×50 grid over $[-2, 2]^2$.

B. Summary of algorithmic findings

Our preliminary study (100 seeds per configuration) established four behaviors of the vanilla KAN in this setting, condensed in Table I: (i) small KANs match or exceed similarly sized MLPs on the separable univariate Duffing residual; (ii) KANs consistently fail to accurately recover the multiplicative Van der Pol interaction, where MLPs scale gracefully; (iii) KAN performance is fragile with respect to grid size, sparsity regularization, and update schedule, with several configurations yielding negative R^2, a sensitivity absent in MLPs [14]; (iv) deeper KAN configurations are unstable under recurrent (BPTT) optimization [15]. These results pose the systems question this paper answers: *does the parameter efficiency KANs exhibit on separable residuals translate into computational efficiency on edge hardware, or does the B-spline formulation introduce execution- and precision-level costs that negate it?*

III. EMBEDDED INFERENCE FRAMEWORK

A. Engine design

We implement both residual branches and the HRPINN cell (Eq. 1) in a dependency-free C++17 engine (libc/libm only, statically linked for minimal Linux images). Both branches are allocation-free, cache-contiguous scalar loops: the MLP is a chain of fused GEMV+ReLU passes over row-major weights; the KAN evaluates, per input feature, the SiLU base path and the Cox–de Boor recursion over that feature's (generally non-uniform, training-adapted) knot vector, followed by the spline GEMV with the per-edge scaler pre-folded into the spline weights at export. Models are exported from PyTorch into a compact binary format carrying weights, knot vectors, the normalizer, the integrator step, and per-tensor calibration ranges.

Two implementation choices matter for fairness. First, the default KAN path reproduces the reference PyTorch numerics operation-for-operation (explicit divisions by knot differences), which lets us pin the engine against the training framework; a second, deployment-optimized path multiplies by reciprocals precomputed at load time (the grid is frozen at inference) and is reported alongside it, so the KAN is measured at its best plausible scalar implementation. Second, both architectures receive identical treatment in every other respect: same compiler and flags (`-O2`, `-ffp-contract=off`), single thread, same measurement harness.

B. Verification methodology

Because a hardware evaluation is only as sound as the equivalence between the measured binary and the published algorithm, the engine is pinned to the training stack by golden-vector suites generated from the trained PyTorch models: 256 branch inputs per model (uniform, spline-knot-boundary, out-of-domain, and on-trajectory states), 64 single cell steps, and three 500-step closed-loop rollouts. The FP32 engine matches PyTorch within 10^{-4} relative on all 20 exported models and within 5×10^{-3} normalized state error over full rollouts; the MLP path is bit-identical to a NumPy reference. The quantized engine (below) is pinned within one quantization LSB against an independent Python implementation of the same specification. The full suite (35 tests, $\approx$11k checks) passes identically when cross-compiled to RV64GC and executed under `qemu-riscv64`, and integer-path outputs are bit-exact across x86-64 and RISC-V; the board workflow re-runs the golden verification on-device before any measurement.

C. Post-training quantization strategy

We implement post-training *static* quantization in its baseline form, deliberately without KAN-specific co-design (which prior work indicates is necessary to keep KAN accuracy losses below fractions of a percent [7]): symmetric per-tensor weight quantization ($\text{scale} = \max|w|/q_{\max}$), affine per-tensor activation and state quantization calibrated from min/max ranges over the training trajectories, at INT16, INT8, and (as additional degradation data points) INT6 and INT4. Execution follows the standard simulated-integer model: values are stored with intN semantics and dequantized for float arithmetic at tensor boundaries; the B-spline bin search consumes the truncated (dequantized) input, reproducing the mis-binning mechanism hypothesized in the literature. MLP biases remain exact, as int32 biases are in real integer pipelines. Consequently, quantized configurations are evaluated for *accuracy and stability* (H3), not speed: we make no latency claims about integer execution. Five quantization modes allow attribution: `full` (state, weights and activations, with the recurrent state re-quantized every step, as a fully integer sensor-to-actuator loop would), `residual_only` (float loop, quantized branch), and `input`-, `weights`-, and `activations`-only.

IV. HYPOTHESES AND EMBEDDED EVALUATION

A. Setup

All 8 configurations of Table II are trained per system (Duffing, Van der Pol) with the BPTT recipe of the algorithmic study (5 seeds; the *median* seed by discovery R^2 is exported, avoiding cherry-picking), yielding 16 primary models plus

TABLE II
EVALUATED RESIDUAL BRANCHES (IDENTICAL TRAINED WEIGHTS IN PYTHON AND C++; MEDIAN-OF-5-SEEDS EXPORT). OPS = SCALAR FLOPS PER BRANCH INFERENCE (ANALYTICAL); EXP = TRANSCENDENTAL CALLS. WEIGHTS INCL. KNOT VECTORS, FP32.

Arch.	Config.	Layers	Params	Ops (+exp)	Bytes
KAN	Very Small	[2,4,1]	120	1350 (+6)	720
	Small	[2,8,1]	240	2322 (+10)	1344
	Wide	[2,16,1]	480	4266 (+18)	2592
	Deep	[2,8,8,1]	880	4986 (+18)	3904
MLP	Tiny	[2,8,8,1]	105	209	420
	Small	[2,16,16,1]	337	673	1348
	Medium	[2,32,32,1]	1185	2369	4740
	Large	[2,64,64,1]	4417	8833	17668

TABLE III
H1 — RESIDUAL-BRANCH LATENCY (NS/CALL, MEDIAN). "RECIP" = KAN WITH PRECOMPUTED-RECIPROCAL BASIS PATH. BOARD COLUMN MEASURED ON THE VISIONFIVE 2. PC: X86-64, GCC 13.3, -O2, SINGLE CORE.

Arch.	Config.	Params	PC	PC recip	U74 (board)
KAN	Very Small	120	244	203	[TBD: ns]
	Small	240	428	372	[TBD: ns]
	Wide	480	763	658	[TBD: ns]
	Deep	880	882–2033†	780–1863†	[TBD: ns]
MLP	Tiny	105	54	—	[TBD: ns]
	Small	337	146	—	[TBD: ns]
	Medium	1185	414–484‡	—	[TBD: ns]
	Large	4417	1549–1712‡	—	[TBD: ns]

†range across the two systems' trained weights (see text); ‡minor spread.

4 TF-trained KAN reference configurations. Latency uses a pinned single core, 10^4 warmup calls, and 3 interleaved rounds of 10 repetitions $\times 10^5$ calls over a fixed buffer of 1024 states, with checksum sinks preventing dead-code elimination; we report medians. Development-host numbers (x86-64, Zen; gcc 13.3) are reported to characterize the architectures; the RISC-V numbers (VisionFive 2, U74 @ 1.5 GHz, gcc 13.3 cross, identical flags) are the paper's headline platform. [TBD: Board latency table pending measurement session; procedure and pre-verified binaries are frozen in the artifact.]

B. H1: Execution latency

Hypothesis. Despite its lower parameter count at comparable Duffing accuracy, the KAN residual branch incurs substantially higher per-step latency than the MLP in scalar execution.

Rationale. Per branch inference, an MLP layer costs $n_{in}n_{out}$ multiply–accumulates; a KANLinear layer costs a base GEMV ($n_{in}n_{out}$), n_{in} SiLU evaluations (each one `expf`), the per-feature Cox–de Boor recursion ($\mathcal{O}((G{+}2k)k)$ data-dependent operations including divisions), and a spline GEMV ($n_{in}n_{out}(G{+}k)$) [8], [7]. Table II makes this concrete: at comparable parameter count (KAN Very Small vs. MLP Tiny), the KAN executes 6.5× the scalar operations plus transcendentals.

Results (development host; board pending). Table III confirms the hypothesis on scalar x86: at comparable parameter count, KAN Very Small (120 params) takes 244 ns per call against 54 ns for MLP Tiny (105 params) — 4.5× — and KAN Small (240) takes 2.9× the latency of the more accurate MLP Small (337). The deployment-optimized reciprocal path recovers 6–18% (median 15%), still leaving a 3.7× gap at matched size; the conclusion is robust to implementation effort. Normalized differently: the KAN spends ≈2.0 ns per parameter versus ≈0.4 ns for MLPs, so the parameter-efficiency argument inverts at execution time. Two further observations strengthen the mechanism account. First, MLP latency tracks its operation count almost perfectly across three orders of magnitude (Table II vs. III), while KAN latency per operation is consistently higher, reflecting the recursion's data-dependent branching and non-contiguous accesses. Second, KAN latency is *weight- and input-dependent*: the two Deep KANs — architecturally identical, differing only in trained weights and adapted grids — differ by 2.2× (882 vs. 2033 ns), a variance MLPs do not exhibit (<10%) and itself a liability for worst-case execution-time analysis in real-time CPS. [TBD: Confirm ratios and the data-dependent effect on the U74; expectation from the in-order pipeline is equal or larger gaps.]

Full accuracy–latency Pareto fronts (every model of Table I against its measured latency) are provided in the artifact; MLPs dominate the front everywhere except the extreme low-memory corner on Duffing, preempting the objection that KANs would win at equal *accuracy* rather than equal parameters.

C. H2: Energy per integration step

Hypothesis. The KAN branch consumes disproportionately more energy per HRPINN step (μJ) than the accuracy-comparable MLP.

Rationale. On a single in-order core at fixed frequency, energy per step is dominated by active-power × latency; H1's latency gap should therefore reproduce in energy, amplified if the KAN's branchy, less cache-regular execution also raises average active power (pipeline stalls keep the core busy without retiring work) [9].

Protocol. Board powered through an inline USB-C power meter; differential measurement (idle power subtracted from steady-state power during a 120 s fixed-duration step loop, with the loop's measured steps/s), so that $\mu\mathrm{J/step} = (P_{\mathrm{load}} - P_{\mathrm{idle}})/(\mathrm{steps/s})$. Governor pinned, idle re-measured at session end (±3% drift bound). The complete pipeline (loop binary, log format, aggregation) is frozen and was validated end-to-end in a dry run; only the meter readings are pending.

Expected/observed results. [TBD: Energy table (μJ/step for all 8 configurations × 2 systems), measured on the VisionFive 2. Expected from H1: KAN μJ/step exceeding matched MLPs by at least the latency ratio; report whether average active power also differs, which would indicate microarchitectural (stall-driven) amplification beyond the latency effect.]

TABLE IV
H3 — CLOSED-LOOP DIVERGENCE (`full` MODE): FRACTION OF 20 ICS DIVERGED / MEDIAN DIVERGENCE STEP (2000 = HORIZON, "—" = NEVER). MATCHED ACCURACY-COMPARABLE PAIRS GROUPED.

		INT16		INT8	
System	Model	div.	step	div.	step
Duffing	KAN Very Small	0 %	—	100 %	**6.5**
	MLP Tiny	0 %	—	100 %	282
	KAN Small	0 %	—	100 %	119
	MLP Small	0 %	—	100 %	316
Van der Pol	KAN Very Small	0 %	—	100 %	78
	MLP Tiny	0 %	—	100 %	196
	KAN Small	100 %	575	100 %	93
	MLP Small	0 %	—	100 %	146

D. H3: Recurrent dependability under quantization

Hypothesis. Under baseline INT8 post-training quantization inside the closed recurrent loop, KAN branches lose trajectory fidelity catastrophically earlier than matched MLPs, whereas INT16 is safe for both.

Rationale. KANs depend on precise positioning of inputs within spline grid intervals, and their per-edge functions have heterogeneous ranges that share single per-tensor scales under baseline PTQ; prior work reports that keeping KAN quantization losses small requires dedicated co-design techniques [7]. In a recurrent integrator, any such localized error is fed back and compounds across steps.

Protocol. For every model: 20 initial conditions, 2000-step (100 s) closed-loop rollouts at FP32/INT16/INT8/INT6/INT4 in `full` and `residual_only` modes plus the three single-point attribution modes at INT8; per-step error is the normalized-state L_2 distance to the same model's FP32 rollout (isolating quantization from model error); divergence is the first crossing of 0.1. Deterministic and bit-reproducible across hosts, so PC results are authoritative; [TBD: two on-board spot checks demonstrating bit-equality, per runbook].

Results. Table IV confirms the hypothesis with an important refinement. At INT16, all matched small models remain within tolerance for the full 100 s horizon (the only failures anywhere at INT16 are KAN configurations already flagged as fragile in Table I). At INT8 in `full` mode — where the state itself is requantized each step — *every* model eventually crosses the threshold, because 8-bit state resolution ($2/255$ of the normalized range) accumulates in any recurrent integrator; the architectures differ radically in *when*: the Duffing KAN Very Small collapses at a median of 6.5 steps (0.3 s) versus 282 steps (14 s) for the matched MLP — $43\times$ earlier — with the same ordering across all matched pairs. `residual_only` mode (float loop, quantized branch) removes the state-truncation floor and preserves the gap: KAN Small diverges at a median step 137 versus 1183 for MLP Small on Duffing, and the best small MLP never diverges where its KAN counterpart fails within seconds.

Mechanism. The attribution experiment (Table V) overturns the most commonly cited explanation. Quantizing *only the input* — the grid mis-binning mechanism — is essentially harmless to both architectures: at INT8 the input step (≈ 0.008) is $50\times$ smaller than a spline bin (0.4). The damage comes from *weight* quantization (KAN trajectory MSE 0.204 vs. 6×10^{-5} for the MLP: three orders of magnitude) and secondarily from activations. The cause is structural: a KANLinear layer concentrates most parameters in one spline-coefficient tensor whose per-edge sub-functions have heterogeneous magnitudes, so a single per-tensor scale crushes the small coefficients that shape individual basis segments; the MLP's homogeneous weight matrices tolerate the same scheme. This redirects mitigation: per-edge (per-channel) spline-weight scales, not finer input grids, are the co-design KANs need — consistent with the hardware-aware quantization emphasis of [7].

TABLE V
H3 ATTRIBUTION AT INT8 (DUFFING, MATCHED PAIR): MEDIAN TRAJECTORY MSE VS. FP32 ROLLOUT WHEN QUANTIZING ONLY ONE COMPONENT.

Quantized component	KAN Very Small	MLP Tiny
input only	3×10^{-6}	2×10^{-6}
weights only	0.204	6×10^{-5}
activations only	0.048	2×10^{-5}
full loop	0.275	4×10^{-3}

Bridge to the discovery metric. Recomputing the algorithmic study's discovery R^2 through the quantized engine collapses the two papers' metrics into one table: on Duffing, KAN Very Small falls from $R^2 = 0.90$ (FP32) to -0.10 at INT8 while MLP Tiny degrades from 0.95 to 0.92; KAN Deep falls to -0.20 while MLP Medium retains 0.94. On Van der Pol, both architectures degrade mildly at INT8 (≤ 0.02), showing the collapse is not universal but weight-distribution-dependent — reinforcing the per-tensor-scale mechanism. Error-growth curves for all models and precisions are in the artifact.

V. DEPLOYMENT GUIDELINES FOR EMBEDDED PHYSICS-INFORMED CPS

1) **Default to MLP residual branches** for dependable edge deployment: at matched accuracy they are faster (H1), [TBD: more energy-efficient (H2, pending board numbers)], latency-deterministic, and degrade gracefully under baseline quantization (H3).
2) **Reserve vanilla KANs for the low-memory, additive-physics corner**: FP32 or INT16 execution, strictly separable residual structure, generous real-time margins (their latency is weight- and input-dependent), and no INT8 pipelines without KAN-specific quantization co-design.
3) **Quantize the loop, not just the network, in your analysis**: in `full`-integer loops even well-behaved MLPs accumulate state-truncation error (all INT8 rollouts eventually drift); dependability budgets must be set against horizon-specific divergence times, which our protocol measures directly.

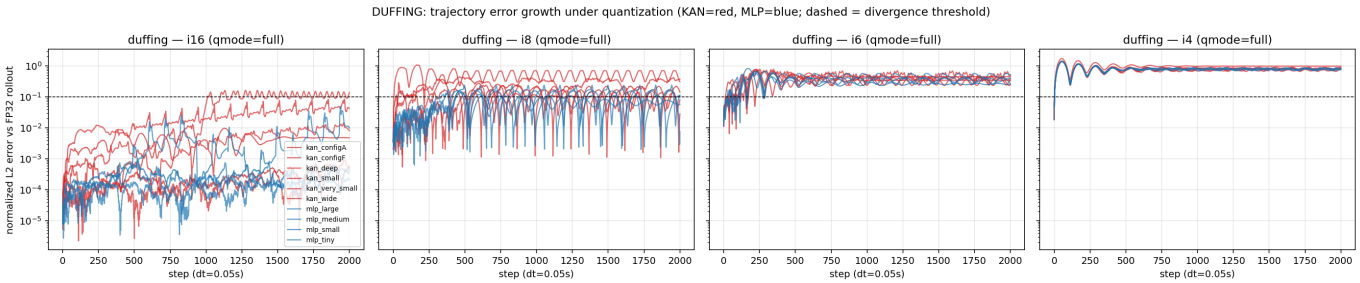


Fig. 1. Trajectory error growth under quantization (Duffing, `full` mode, error vs. the same model's FP32 rollout; KAN models red, MLP blue; dashed line = divergence threshold). INT16 keeps all models within tolerance; at INT8 the KANs cross the threshold one to two orders of magnitude earlier than matched MLPs; INT6/INT4 destroy both. [TBD: Re-render with final styling and the Van der Pol panel if page budget allows.]

4) **If KANs must be quantized**, prioritize per-edge spline-weight scales over input-grid precision — our attribution data shows the per-tensor weight scale, not bin misassignment, is the failure driver.

VI. Threats to Validity

Internal. The engine is pinned to the training framework by golden tests (Section III-B); measurement noise is bounded by interleaved rounds and reported percentiles. The KAN is measured at both the reference-faithful and a deployment-optimized basis implementation, bounding implementation bias at $\leq$18%. Exported models are median-of-5 seeds. **Construct.** Quantized execution is simulated-integer (standard PTQ semantics); we therefore make no claims about integer *latency*, only about precision-induced behavior, which the simulation reproduces exactly by construction. Energy is measured differentially at the board input, capturing SoC+DRAM but not isolating the core — appropriate for system-level budgeting. **External.** Two canonical two-state plants and one scalar RV64GC core; Section I-A argues why this is the right controlled setting and in which direction each simplification is conservative. Generalization to vectorized cores (RVV, NEON) is explicitly out of scope: vector units would change both architectures' constants, though the KAN's gather-heavy, branchy recursion vectorizes worse than GEMV in principle.

VII. Conclusion and Future Work

We bridged an algorithmic result — when vanilla KANs match MLPs as hard-constrained residual learners — to its systems consequence on open embedded hardware, with a verified, released pipeline in which every measured binary is provably the published algorithm. The parameter efficiency of the vanilla KAN does not survive deployment scrutiny: at matched accuracy and parameter budget it executes several times slower under scalar execution, with weight-dependent timing variance, and its quantization robustness in the recurrent loop is dominated by a structural weakness (per-tensor spline-weight scales) that baseline PTQ cannot accommodate — while MLPs remain fast, deterministic, and gracefully degrading. [TBD: One-sentence quantitative summary of board latency and energy once measured.]

Future work follows three axes: (i) evaluating KAN variants and co-designed quantization (per-edge scales, LUT-based basis evaluation, CIM-style analog activation generation [16], [7]) against this baseline; (ii) scaling the dependability methodology to higher-dimensional and eventually industrial plants (vehicle dynamics, power-electronics twins) now that the architectural variables are characterized in isolation; (iii) vectorized (RVV) implementations to test whether the scalar gap narrows or widens with data-level parallelism.

References

[1] E. N. Spotorno, A. A. Fröhlich *et al.*, "Hard-constrained neural networks with physics-embedded architecture for residual dynamics learning and invariant enforcement in cyber-physical systems," *arXiv preprint arXiv:2511.23307*, 2025.

[2] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, and M. Tegmark, "Kan: Kolmogorov-arnold networks," *arXiv preprint arXiv:2404.19756*, 2024.

[3] B. C. Koenig, S. Kim, and S. Deng, "Kan-odes: Kolmogorov–arnold network ordinary differential equations for learning dynamical systems and hidden physics," *Computer Methods in Applied Mechanics and Engineering*, vol. 432, p. 117397, 2024.

[4] K. Ma, X. Lu, N. L. Bragazzi, D. Selzer, T. Lehr, and B. Tang, "Integrating kolmogorov-arnold networks with ordinary differential equations for efficient, interpretable, and robust deep learning: Epidemiology of infectious diseases as a case study," *Infectious Disease Modelling*, 2025.

[5] N. A. Daryakenari, K. Shukla, and G. E. Karniadakis, "Representation meets optimization: Training pinns and pikans for gray-box discovery in systems pharmacology," *Computers in Biology and Medicine*, vol. 201, p. 111393, 2026.

[6] W. Liu, K. Bacsa, L. C. Tang, and E. Chatzi, "Structured kolmogorov-arnold neural odes for interpretable learning and symbolic discovery of nonlinear dynamics," *arXiv preprint arXiv:2506.18339*, 2025.

[7] W.-H. Huang, J. Jia, Y. Kong, F. Waqar, T.-H. Wen, M.-F. Chang, and S. Yu, "Hardware acceleration of kolmogorov-arnold network (kan) for lightweight edge inference," in *Proceedings of the 30th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2025.

[8] P. Khalid, "Placeholder: analytical operation-count comparison of kans vs mlps," 2025, **TO BE LOCATED AND VERIFIED**.

[9] P. Zhang, "Placeholder: hardware implementation cost of kan basis functions," 2025, **TO BE LOCATED AND VERIFIED**.

[10] F. Mostajeran and S. A. Faroughi, "Scaled-cpikans: Spatial variable and residual scaling in chebyshev-based physics-informed kolmogorov-arnold networks," *Journal of Computational Physics*, vol. 537, p. 114116, 2025.

[11] D. W. Abueidda, P. Pantidis, and M. E. Mobasher, "Deepokan: Deep operator network based on kolmogorov arnold networks for mechanics problems," *Computer Methods in Applied Mechanics and Engineering*, vol. 436, p. 117699, 2025.

[12] Z. Zhang, Q. Wang, Y. Zhang, T. Shen, and W. Zhang, "Physics-informed neural networks with hybrid kolmogorov-arnold network and augmented lagrangian function for solving partial differential equations," *Scientific Reports*, vol. 15, no. 1, p. 10523, 2025.

[13] StarFive Technology, "Starfive visionfive 2 single-board computer, jh7110 soc (sifive u74, rv64gc)," 2023, https://www.starfivetech.com.

[14] A. Noorizadegan, S. Wang, and L. Ling, "A practitioner's guide to kolmogorov-arnold networks," *arXiv preprint arXiv:2510.25781*, 2025.

[15] S. Sohail, "On training of kolmogorov-arnold networks," *arXiv preprint arXiv:2411.05296*, 2024.

[16] P. Wen, "Placeholder: Gaussian-like analog compute-in-memory activation cell," 2025, **TO BE LOCATED AND VERIFIED**.